

Technical Documentation

- Game Development for Publishing
 - Game UI and Preloader
 - Max Win Limitation
 - Replay
 - Using of Casino FreeBets
 - Features of Bonus Game Implementation
 - Requirements
 - Frontend Events Sending
 - Required Game Events:
 - Game Translation and Localisation
 - Why is this important?
 - List of supported locales
- Introduction in Technical Implementation
 - Glossary
 - Description of the 'game function' model for Hyper Hive
 - Advantages of 'game as a function' approach
- Game Flow Description
 - Workflow
 - init method
 - info method
 - play method
 - Step, Round, Transaction
 - Buy bonus
- Development overview
 - Local runner
 - Example
 - Deployment
 - GodMode
 - Used during game development, testing and debugging.
 - dev_runner
 - Freebets
 - Freebets in local development runner
 - Multi tab protection (state_lock)
- API Description
 - Runner API
 - Methods
 - "method": "init"
 - "method": "info"
 - "method": "play"
 - "method": "replay"
 - API Error handling:
 - Game API
 - "method": "play"
 - Best practices for game / round state
 - RNG API
 - "method": "rand"
 - "method": "rand_range"
 - "method": "sample"
 - "method": "shuffle"
- Additional requirements from Game Provider

Game Development for Publishing

Game UI and Preloader

All games developed for the publishing platform must use the BGaming game UI and a preloader designed according to a specific format, containing:

- Game logo
- Provider studio logo
- BGaming company logo

The link to the requirements for using the game UI and preloader is provided below.

<https://drive.google.com/file/d/1DHe8rguRSduM-swqtqMvWkwjuqhXVx6Mi/view?usp=sharing>

Max Win Limitation

Each game must have specified Max Multiplier. The win in single round should not be higher than (Base Bet x Max Multiplier).

If game generate outcome higher that max multiplier allowed, for example in the middle of bonus game, game process should be terminated, round should be closed and player awarded with win accumulated on this moment. Accumulated win should be capped to value (Base Bet x Max Multiplier).

Example of popup of max win reach.



Replay

An essential feature of the game is the replay function. The replay is described in the technical documentation below ([link](#)).

From the game's perspective, the replay works as follows:

- The runner returns the round to the game.
- Based on the received information, the game must reproduce the entire round with the ability to:
 - Start
 - Stop
 - Move to each specific step played within the game round.

Examples of replay implementations for a regular round and a round where a bonus game was triggered are provided under the links below

[Link for replay example, regular round](#)

[Link for replay example, bonus game](#)

Using of Casino FreeBets

This document provides a description of the FreeBets feature. Supporting of casino freebets is a mandatory part of game development.

Using FreeBets is an important tool for promoting the game on the market.

Accordingly, FreeBets Casino must be integrated into the game.

Features of Bonus Game Implementation

The bonus game must be implemented in a way that divides the round into ****steps****. We should not receive financial information only at the last step of the bonus game—financial information must be received at every step.

This is necessary for the following reasons:

- If the connection is lost or other issues arise, the player can return and resume the game from the same step of bonus play.
- Technical closure of unfinished bonus rounds:
 - If a bonus round remains unfinished for 24 hours, we must:
 - Deduct the bet that the player placed to start the bonus round.
 - Credit the player with all winnings they accumulated in intermediate steps.

Without financial information for each step, we cannot properly close the bonus game, which could result in a situation where the player's stake is deducted without awarding any winnings. This is unacceptable for most casinos and may lead to the game's removal.

Requirements

- The bonus game must be clearly divided into steps.
- Financial information must be received for each step to accurately track winnings, the moment they are received, and the step on which they were awarded

Frontend Events Sending

Game should send frontend events based on [documentation](#)

Events must be sent using window.dispatchEvent with the corresponding data payload.

Example:

```
const event = new CustomEvent(event_name, {detail: {event_name, payload}})
```

```
window.dispatchEvent(event)
```

Required Game Events:

event_name	description	trigger	payload
lifecycle:game:initialized	The game is initialized (all resources are loaded, and everything required for display is created in memory). At this point, the game UI appears, and we initialize the display of Growth product UI components.	The game code is initialized (the game graphics will be rendered in the next frame).	-
round:round:started	The round has started (a round is defined as a spin or a series of spins sharing the same round_id).	Click on Spin button	round: { isAutoSpin: boolean; // Spin in Autospin mode isFreeSpin: boolean; // Freespin spin isFreeRound: boolean; // Freebet spin isRespin: boolean; // Respin };
round:round-animations:finished	The round has ended (after all animations, including win animations, free spins exit popups, etc., have completed).	The round has ended (after all animations, including win animations, free spins exit popups, etc., have completed).	-
player:balance:updated	Balance updated on UI	Balance updated on UI	player: { balance: number; // Player balance on UI (cents) currency: string; // Currency Code (USD, EUR, etc.) };
autospin:session:started	Autospins started	Player turned Autospis on	autospins: { isActive: boolean; // Autospins activeted count: number 'infinity'; // Amount of autospins selected by player remaining: number; // Remained autospins };
autospin:session:stopped	Autospins stopped	Autospins were ended or stopped by player	-

ui:game-area: covered	The game field is currently covered/overlapped by other element (e.g. game settings, popups, modals, errors, etc.). When this event is triggered, we hide the Growth product UI.	One of the following is currently open: game settings, popups, modals, errors, etc.	-
ui:game-area: uncovered	The game field is no longer covered/overlapped. When this event is triggered, we show the Growth product UI.	One of the following has been closed: game settings, popups, modals, errors, etc.	-

Game Translation and Localisation

As long-standing providers in the market, we recognize that localisation significantly enhances the game's promotion potential.

It also affects how many casinos across different markets will be willing to include the game in their portfolio.

We strongly recommend localising games. However, even if the game provider decides to release the game only in English, it is crucial to adapt the game to social English.

The link to the social English glossary is provided below.

Why is this important?

- This translation requires minimal effort and resources from the game provider.
- It allows access to a broad range of markets.
- Adding social English will provide a significant competitive advantage for the game.

List of supported locales

Country Name	Code
Thai	th
Vietnamese	vi
Korean	ko
Danish	da
Dutch	nl
Russian	ru
Ukrainian	uk
Indonesian	id
French	fr
Bulgarian	bg
Italian	it
Spanish	es
Romanian	ro
Swedish	sv
Chinese	zh
Turkish	tr
Greek	el
Brazilian Portuguese	pt-BR
German	de
Social English	en-SOCIAL

Introduction in Technical Implementation

Glossary

hyper hive - publishing platform, that unified collaboration between games and casino platforms;

game provider - game development studio;

runner - hyper hive service which is working with game frontend;

game frontend - client side of the game implemented by **game provider**;

game function - game backend implemented by **game provider**;

backoffice - hyper hive backoffice used for game and tables configuration.

Description of the 'game function' model for Hyper Hive

A classic client-server web application consists of:

1. frontend - web page
2. backend - a service that receives requests from the frontend, stores game state, interacts with other services, store private logic.

The application from the game provider created for hyper hive consists of the same parts, but its backend takes on only the task of calculating the winnings, and delegates the storage of the state, the generation of random numbers and the interaction with the casino and other delegates to the runner service.

Player <-> Hyper Hive Runner <-> Game Function <-> Hyper Hive Runner <-> Casino

Runner sends the latest state of the game, required settings, etc. on every request to the game's backend. In this case, the payoff calculation on the game side does not need any third-party information and is written as a pure function (without side effects), so we call the request handler and the backend of the game itself a game function

Advantages of 'game as a function' approach

- The implementation of the game makes easier, all infrastructure logic is already implemented in runner;
- Transparency when logging and debugging game decisions. There are only input parameters and output result;
- Easier testing and reproducing of exact game behavior;
- The game has no access anywhere and does not ask for anything, which has a positive impact on security;
- It is not possible to call game methods bypassing runner, it also has a positive effect on security;
- Analysis of the legitimacy of write-offs is simplified, because the connection of user actions with game function decision making is unambiguous;
- Simplifies the calculation of the statistical qualities of the game (ex. RTP).

Game Flow Description

Game frontend using two runner's methods:

- **init** - called on game initialization to get the current state of the game;
- **info** - called when it is necessary to get the current balance and freebets issues;
- **play** - called when the player has performed an action (the game function must implement the play method.)

Workflow

1. The casino sends a request to launch the game in hyper hive
2. Hyper hive generates a link and gives it to the casino, the casino transfers generated link to the player
3. The game frontend requests the init method to get the balance, the list of available bets, and other game settings
4. The player places a bet and the game frontend requests the play method on runner
5. The body of the request for runner is a black box that it passes to the game function
6. Game function processes the request and returns a response in a specific format. The response contains information about the changed state of the game, stakes, winnings and the response to be returned to the request
7. Runner saves the state returned by the game function, makes decisions about sending transactions
8. Runner returns a response intended for the game frontend (part of the game function response)
9. Game frontend receives this response and displays the result to the player.

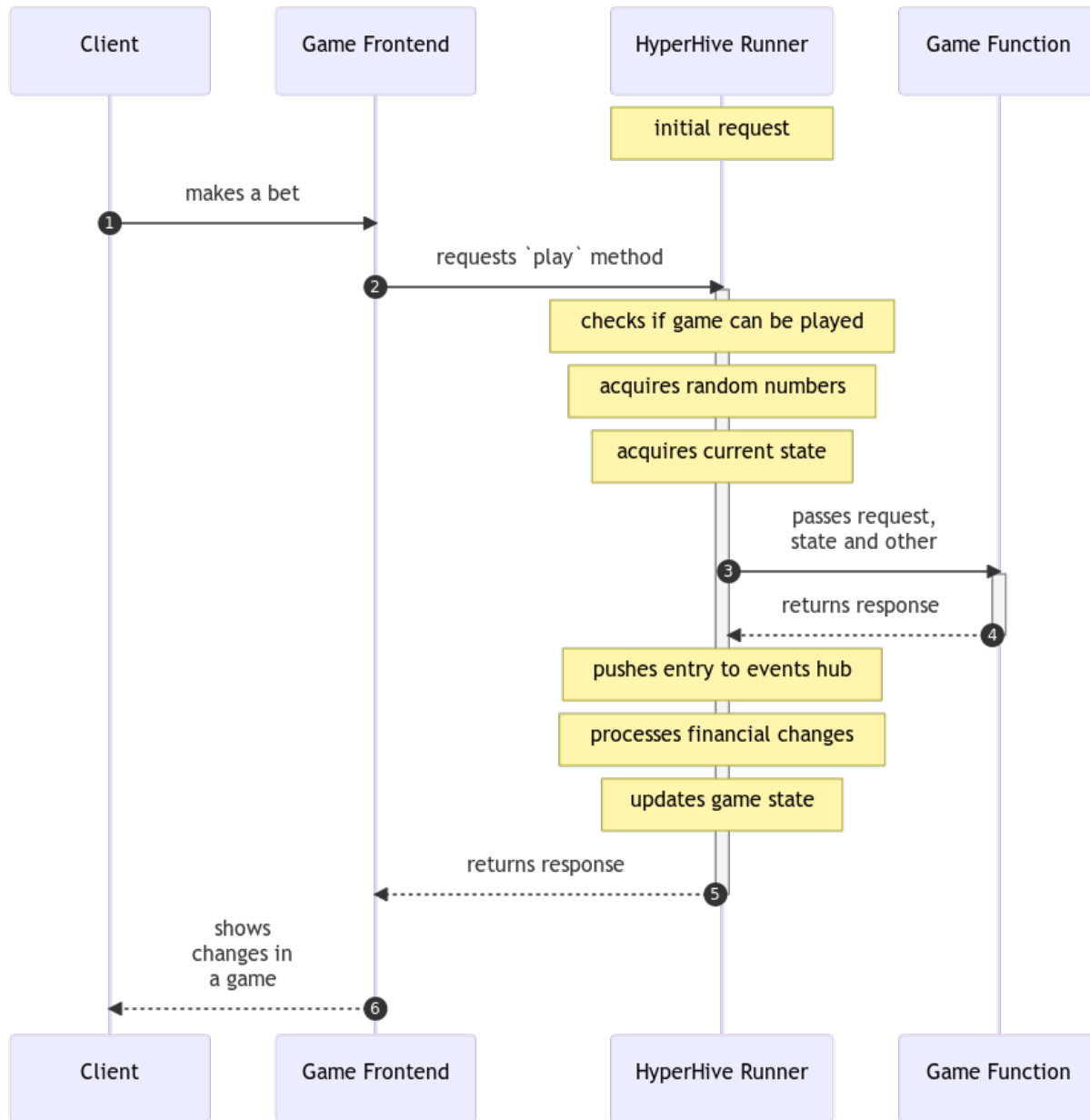
init method

Returns the current state, user balance and game settings.

info method

Returns the current balance and freebets issues.

play method



1.

- Game frontend sends a request to runner in response to user action.

There are two fields in the parameters of the play method:

token - his game frontend gets from his page url

req is a json object whose body is not regulated by the runner, it is a black box for it, it will pass it unchanged to the game function.

- runner by token requests and prepares parameters for the game function.

game is the state of the game that the game saved in the previous step.

round - similar to the game parameter, it also stores the state, but is reset when the round closes.

req - the body of the request that runner received from the game function

config is a compiled configuration containing game settings, table.

The config field contains the game settings. They are assigned in the backoffice (games, tables, limits) in case of a defield in hyper hive or passed with the --config parameter in case of a local runner.

- game function receives parameters prepared by runner and makes decisions:

Save anything in state to use in later steps by returning an updated value to the game field.
Do the same with round, but here we store the state associated with the accumulated winnings (amount, modifiers, etc.) relevant within the round.
Add an entry to the finance list indicating the amount of money won.
Close round or not (see next section)

- Runner get the result, do the following:

Writes the returned value and parameters to the event hub to keep history.
If necessary, sends withdrawals or accumulated deposits to the casino
Updates the game state from the game and gold fields

- Game frontend from everything returned by the game function receives only the values from the resp field intended for it.
- The game frontend updates the ui for the user based on the response.

Подробности формата запроса и ответа методов смотри в [API](#).

Step, Round, Transaction

Step - is the execution of the play method.

Round - is a set of steps. In a simple case, a round consists of one step. The round closes at the step where the game function returns final to true.

The user's funds are written off at the same step in which the game function indicated this in the finance field. The payout is credited at the end of the round.

An example of rounds of several steps would be gamble in slots (the user does not automatically credit himself with a win, but must also decide whether he will risk it or not) or poker.

Buy bonus

To trigger bonus round:

1. Game Frontend sends play with purchased_feature: req = {bet: 10, purchased_feature: "buy_bonus"}
2. Game Backend replies with finance: [{type: "betting", bet: -10 * bonus_multiplier, base: 10}]
3. Profit! Round will be marked as "bonus"

Requirements:

- Game front should hide "buy bonus" buttons if 'buy_bonus' not specified in config.purchased_features array
- Game front should send correct req.purchased_feature in play request to start round
- Game backend should calculate bonus round price based on req.bet & req.purchased_feature and reply with correct finance[].bet = req.bet * bonus_multiplier

Development overview

In order to start development, you need to get a local version of runner to work on a local machine. Please contact [Igor Bondarenko](#) for it.

Here is the pseudo code examples.

Local runner

Launch local runner:

```
runner -p 4000 -c ./myGame/config.yml -b http://localhost:3050/myGame/backend
```

Game run on port 4000, and game function will be available here: <http://localhost:3050/myGame/backend>

Local runner exposes Web UI on <http://localhost:4000> by default where you can manage sessions & freebets.

Example config:

config.yml

```
---
game:
  code: myGame

backend:
  url: "http://localhost:4051/backend"

sessions:
  fda1091b1c7349e8dbcc4c2d6fd89926:
    state: active
    currency: usd
    locale: en
    bet_limits: [100, 200, 300, 400, 500]
    freebets_limits: [100, 200, 300, 400, 500]
    default_bet: 300
    purchased_features: ["buy_bonus", "buy_chance", "buy_bonus_and_chance"]
    balance: 100000
    rtp: "95.5"

  e7f0110f933c6da531ab34fa31b408b7:
    state: closed
    currency: usd
    locale: ru
    bet_limits: [100, 200, 300, 400, 500]
    freebets_limits: [100, 200, 300, 400, 500]
    default_bet: 300

  8390e97937c36a39209c7b537a72c76f:
    state: inactive_player
    currency: usd
    locale: de
    bet_limits: [100, 200, 300, 400, 500]
    freebets_limits: [100, 200, 300, 400, 500]
    default_bet: 300

urls:
  return_url: https://google.com
  deposit_url: https://binance.com
  history_url: https://history.com

# Additional/overridden currencies
currencies:
  usdt:
    subunits: 100000000
    exponent: 2
```

Example

Let's create an html page that will act as our game frontend.

The token is always passed to our page in the url, which we use in all interactions with the runner.

./myGame/frontend/index.html

```

RUNNER_URL = 'http://localhost:4000'
TOKEN = new URLSearchParams(...).get('token')

callRunnerMethod(rpc_method, params, handler) {
  data = {jsonrpc: "2.0", method: rpc_method, params: params};
  post(RUNNER_URL, data).then(handler)
}

// When loading the application, we obtain the information necessary for display.
onLoad() {
  callRunnerMethod("init", { token: TOKEN }, (response) => {
    print("Balance: ", response.data.result.balance)
    print("Bets count: ", response.data.result.state["bets_count"] || 0)
  })
}

// When you click on the button to place a bet
onBetButtonClicked() {
  callRunnerMethod("play", { token: TOKEN, req: {anything: "could be anything but only bet is required", bet:
10} }, (response) => {
    result = response.data.result

    print("Won: ", result.resp.player_message)
    print("Balance: ", result.balance)
    print("Bets count: ", result.state.resp.bets_count)
  })
}

```

Let's create a game function server and a handler for the play method.

`./myGame/backend/server.js`

```

PORT = 80

// Random numbers are 32 bit unsigned integers
u32_max = 4_294_967_295
u32_half = floor(u32_max / 2)

is_win(random_number) { return random_number > u32_half; }

RPCMethods = {
  play: function (rpc, params) {
    bet = params.req.bet
    anything = params.req.anything

    if (bet != 10 || !anything) {
      rpc.error('Bad params')
    }
    // Call RNG service using the instructions below to get random number
    random_number = <RNG number>
    //
    bets_count = (params.state.bets_count || 0) + 1

    if is_win(random_number) {
      rpc.response({
        resp: {player_message: "You won!", bets_count: bets_count}
        game: {bets_count: bets_count},
        finance: [{type: "bet", amount: -bet}, {type: "win", amount: bet * 2}],
        final: true // Close round
      })
    } else {
      rpc.response({
        resp: {player_message: "You loose!", bets_count: bets_count}
        game: {bets_count: bets_count},
        finance: [{type: "bet", amount: -bet}],
        final: true // Close round
      })
    }
  }
}

http.createServer(function (request, response) {

  if (request.method == 'POST' request.path == '/api') {
    new RpcService(request, response, RPCMethods);
  } else {
    raise "unsupported"
  }

}).listen(PORT);

```

Here, the game function with a 50% chance gives out a win in the amount of the bet. Also, to illustrate how the state of the game can be saved between requests, we display the number of bets that the player made in this game session (the duration and opening / closing of the game session depends mostly on the casino, but hyper hive can also forcibly close it).

Deployment

After completing the implementation and gaining access to the docker repository, you can send images with the game to hyperhive.

Both servers should run the default http server on port 80.

```
cd /myGame/backend
docker build .
docker image push <hyper-hive-registry-host>/<game provider name>/game-function:latest

cd /myGame/frontend
docker build .
docker image push <hyper-hive-registry-host>/<game provider name>/game-frontend:latest
```

GodMode

Used during game development, testing and debugging.

With each request from HyperHive to the game function, the `god_data` value is taken from a special queue and passed in the parameters of the `play` command in addition to the request from the game frontend and other parameters.

The backend of the game uses this parameter (for example, substituting random numbers or the state of the game) in accordance with the game logic and returns the answer. You can add data to the queue for several consecutive requests at once, so you can implement the desired sequence of game events.

NOTE: We set `ENABLE_GOD_MODE=1` environment variable for backend container in dev/stage environments. If it's not set game should ignore any passed `god_data`.

dev_runner

Use *"God mode"* button in dev runner interface next to needed session to replace god data for it. Just paste the data in JSON format into the form field in the modal window.

Freebets

To support freebets game must:

- Game frontend should make `info` RPC call and show user info about current freebets from `freebets` field. Info about freebets also returned from `play` RPC call.
- If freebets are available to user, game frontend should include `'bet_type: 'freebet'` into `req` object to `play` RPC call. In this case round will be counted as freebet round. Otherwise error will be returned.
- If freebets are available `bet` value in `req` object should be calculated as `bet = config.freebets_limits[freebets.bet_level]` from values received in `init` and `info` rpc calls.

Freebets in local development runner

`runner_cil` exposes management Web UI on <http://localhost:4000> where you can issue freebets for your game session.

Multi tab protection (state_lock)

Algorithm is simple:

1. Each `'init'` and `'play'` response contains field `'state_lock'`
2. The game frontend saves it in the variable
3. The game frontend sends `'state_lock'` field with the next `'play'` request containing the saved value.
4. If state lock differs from the one in the runner/platform - the request will be rejected with an error.
- 4.1. The game frontend should show error to the player and reload the page to restore state.

API Description

Interaction with the server parts of the game engine is based on JSONRPC2 specification (<https://www.jsonrpc.org/specification>) and has a common request structure:

```
{
  "jsonrpc": "2.0",
  "id": string,      // request id
  "method": string,  // name of RPC-method
  "params": object   // input parameters of RPC-method
}
```

Positive response:

```
{
  "jsonrpc": "2.0",
  "id": string,
  "result": object
}
```

Response with error:

```
{
  "jsonrpc": "2.0",
  "id": string,
  "error": {
    "code": integer,    // error code
    "message": string,  // error message
    "data": object      // error details
  }
}
```

Next, we will describe the API as an enumeration of methods with a description of their input parameters and possible response structures.

Runner API

POST https://<RUNNER_HOST>/api

Methods

"method": "init"

Getting information about the configuration and state of the game for a specific session. May change during the course of the game.

-> params:

```
{
  "token": string // session token
}
```

<- result:

```

{
  "balance": integer,      // player balance
  "currency": string,      // table currency
  "currency_attributes": {
    "code": string,        // currency's ISO code (the value is equal to the value of the "currency" field
above)
    "symbol": string | null,      // currency's ISO symbol
    "exponent": integer,      // min number of decimal places
    "subunits": integer      // power of 10 indicating the number of decimal places
  },
  "locale": string,        // session locale
  "freebets": {            // information about free bets from casino if issued, else null
    "bet_level": integer,      // Index of freebets bet value in freebets_limits, eg: bet = config.
freebets_limits[freebets.bet_level]
    "issued": integer,        // amount of issued free bets
    "done": integer,          // amount of already played free bets
    "total_win": integer      // total win with free bets
  },
  "config": {              // game configuration, could be changed during the round
    "rtp": string | null,      // use default if null
    "displayed_rtp": string | null, // use default if null
    "bet_limits": [integer],    // list of possible bets
    "default_bet": integer,      // default bet
    "freebets_limits": [integer] // list of bets allowed for free bets,
    "purchased_features": ["buy_bonus" | "buy_chance" | "buy_bonus_and_chance"], // list of enabled purchased
features for session
  },
  "state": {               // response from previous step, if any, else null
    "round": string,        // round ID
    "resp": object          // response from game function
  },
  "urls": {
    "return_url": string | null, // url user will be redirected after playing session finished, can be null
    "deposit_url": string | null, // url user will be redirected in order to deposit, can be null
    "history_url": string | null, // url user will be redirected in order to view rounds history, can be null
  },
  "state_lock": string      // unique state token for multi tab protection
}

```

<- error:

```

{
  "code": 51100,
  "message": "Something went wrong. Refresh the page."
}

{
  "code": 51300,
  "message": "You are banned. Please contact administrator."
}

{
  "code": 51300,
  "message": "Game is removed from catalog. Please contact administrator."
}

{
  "code": 51300,
  "message": "Something went wrong. Please contact administrator."
}

```

"method": "info"

Getting information about balance and freebets issues

-> params:

```
{
  "token": string // session token
}
```

<- result:

```
{
  "balance": integer,      // player balance
  "freebets": {            // information about free bets from casino if issued, else null
    "bet_level": integer,  // bet index freebets_limits
    "issued": integer,     // amount of issued free bets
    "done": integer,       // amount of already played free bets
    "total_win": integer   // total win with free bets
  }
}
```

<- error:

```
{
  "code": 51200,
  "message": "Something went wrong. Relaunch the game with new session."
}

{
  "code": 51300,
  "message": "You are banned. Please contact administrator."
}

{
  "code": 51300,
  "message": "Game is removed from catalog. Please contact administrator."
}

{
  "code": 51300,
  "message": "Something went wrong. Please contact administrator."
}
```

"method": "play"

Step execution in terms of the round.

-> params:

```
{
  "token": string,          // session token
  "req": {                  // request parameters to game function, should contains `bet` &
    `bet_type` field
    "bet": integer,         // The `bet` must be an integer or it will be explicitly casted to
integer.
    "bet_type": "bet" | "freebet", // The `bet_type` must be "freebet" if there are any freebets available
to user
    "purchased_feature": "buy_bonus" | "buy_chance" | "buy_bonus_and_chance", // Round will be marked as
"bonus" if present, see more in "Buy bonus" section
    "custom_field": "custom_value" // `req` object may contain custom fields which will be passed to game
backend
  },
  "state_lock": string      // unique state token for multi tab protection (obtained
from the previous request play\init)
}
```

<- result:

```
{
  "round": uuid,          // id of current round
  "step": uuid,          // step id
  "balance": integer,     // player balance
  "freebets": {          // Free bets info explained in examples above
    "bet_level": integer,
    "issued": integer,
    "done": integer,
    "total_win": integer
  },
  "resp": object,        // game function response
  "final": bool,         // round end indicator
  "state_lock": string    // unique state token for multi tab protection
}
```

<- error:

```
{
  "code": 51100,
  "message": "Something went wrong. Refresh the page."
}

{
  "code": 51101,
  "message": "There is active freebet. Refresh the page."
}

{
  "code": 51102,
  "message": "There is no active freebet. Refresh the page."
}

{
  "code": 51103,
  "message": "You have insufficient funds."
}

{
  "code": 51200,
  "message": "Something went wrong. Relaunch the game with new session."
}

{
  "code": 51300,
  "message": "Something went wrong. Please contact administrator."
}
```

"method": "replay"

Getting game round data.

-> params:

```
{
  "id": uuid // round id
}
```

<- result:

```
{
  "steps": Step[], // List of round steps in a structure similar to the result play.
  "currency": string, // currency code
  "balance": integer, // balance at the beginning of the round
  "currency_attributes": {
    "code": string, // currency's ISO code (the value is equal to the value of the "currency" field
above)
    "symbol": string | null, // currency's ISO symbol
    "exponent": integer, // min number of decimal places
    "subunits": integer // power of 10 indicating the number of decimal places
  }
}
```

API Error handling:

- generic connection errors(Connection closed, Timeout, etc) - translate as generic error "An error has occurred."
- http statuses other than 200 - translate as generic error "An error has occurred."
- JSONRPC error codes:
 - from -32700 to -32000 inclusive(it's internal server errors described in [JSONRPC specification](#)) -translate as "Woops, an error has occurred. Please contact support referring to the error code {code}."
 - other JSONRPC error codes
 - if message field is present - show message field as is
 - if message field is missing -translate as "Woops, an error has occurred. Please contact support referring to the error code {code}."
- If play method returns JSONRPC error code 51103 and config.urls.deposit_url is present - additionally show "Deposit" button in the modal with error

Game API

POST https://<GAME_BACKEND_HOST>/backend

"method": "play"

Game function call.

-> params:

```
{
  // Contents of the per-round internal state storage.
  // For the first step of a new round: there will be empty object: `{}`.
  // For a not-first step: there will be object that the previous `play` request returned in the `round` field.
  "round": object,

  // Contents of the per-session internal state storage.
  // For a playing session: there will be empty object: `{}`.
  // In all other cases: there will be object that the previous `play` request of the current session returned
  in the `game` field.
  "game": object,

  // request parameters from the game client
  "req": object,

  "config": {
    // actual game configuration
    "rtp": string, // selected RTP for current session
    "purchased_features": string[] // Enabled purchased features for current game
    "gamble_limit": float | null // Max multiplier from init bet allowed for gamble step;
    // eg. for init_bet = 10 and gamble_limit = 2.5 gamble game should not be
    triggered if player can win more that 25 in gamble round
    // value == null means there is no limits
    // value == 0.0 effectively means that
    gamble game should never be triggered
  },
  "god_data": object // data for substituting the response fields from the function (for more
details, see the God Mode section)
}
```

<- result:

```

{
  // Contents of the per-round internal state storage. It can be any json object, it's passed/stored as is.
  // It will be passed as is in the `round` field to the next `play` request(next step) in the current round
  (but not in a new round).
  // It will not be passed to the frontend.
  //
  // Use it to save generic round state that you need pass between steps in the single round, for example:
  //   - base bet value in a multi-step round
  //   - data on "sticky" symbols that affect only current round
  //   - data required for a gamble step(double win or lose all)
  //   - ...
  //
  // See best practices below.
  "round": object,

  // Contents of the per-session internal state storage. It can be any json object, it's passed/stored as is.
  // It's persisted between rounds in a single playing session.
  // It will be passed as is in the `game` field to the next play request in the current session(whether it's a
  new round or the current round).
  // It will not be passed to the frontend.
  //
  // Use it to save cumulative progress/counters that persist for entire playing session, for example:
  //   - total number of special points/coins collected by the player for the current session
  //   - data on "sticky" that affect multiple rounds
  //   - ...
  //
  // If your game doesn't have such mechanics - just return empty object here: `{}`
  //
  // See best practices below.
  "game": object,

  "finance": [          // list of financial transactions
    {
      "type": enum,      // record name, default is: "betting"
      "bet": integer,    // the amount that the casino will charge the player in this round (always negative)
      "base": integer,   // the base from which the payoff is calculated (always positive)
      "win_change": integer // payoff changes (can be negative)
    }
  ],

  // Game function response to be passed to the game frontend (can be any object)
  // This field will be passed as is to the game frontend as `resp` field.
  // Also array of all `resp` objects from one round is returned in the `replay` request which can be used to
  show a replay of the round.
  //
  // Only return data that absolutely necessary to correctly render round result in the frontend
  "resp": object

  // Return true if the current step of the round is final and round is completed/finished.
  // This flag will be passed to the game frontend as `final` field.
  "final": bool,
}

```

<- error:

```

{
  "code": 50000,
  "message": "Game Error",
  "data": {
    "reason": string    // error message
  }
}

```

Best practices for game / round state

- store only those data that absolutely necessary to calculate result of the next step in the current round(`round` field) or the next rounds in the current session (`game` field)

- do not store redundant or meaningless data
 - do not return custom `isFinished` field inside `game/round/resp` objects, use existing `final` flag
 - do not return `null` values for fields where `null` can be replaced by the absence of the field
 - For example: instead `{fieldA: 1, fieldB: null, fieldC: null}` use `{fieldA: 1}`
- do not rely on the order of keys in a json object:
 - JSON standard doesn't define the order of keys in an object, different programming languages/platforms/runtimes can change the order of keys in an object
 - use an array when the order is relevant: instead `{a: 1, b: 2}` use `[[{key: "a", value: 1},{key: "b", value: 2}]`
- do not store json values encoded as the string
 - For example: instead `{"myData": "{\"key\": \"value\"}"}` use plain structure `{"myData": {"key": "value"}}`

RNG API

POST <RNG_URL>

- for `dev_runner` by default it's `http://localhost:4002/api`
- in production we pass full url to the `jsonrpc` endpoint via environment variable `RNG_URL` to backend container, for example `RNG_URL=http://mg-service:4009/api`

"method": "rand"

Generate requested amount of random numbers.

Params

```
{
  "qty": integer // count of random numbers
  "type": enum   // type of random numbers, valid values: <"u32", "f32"> (by default "u32")
}
```

Response

[number] // array of random numbers

Request Example

```
// ->
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "method": "rand",
  "params": {
    "qty": 3
  }
}

// <-
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "result": [2985189278, 1294157622, 3712006809]
}
```

"method": "rand_range"

Generate random numbers from specified range.

Params

```
{
  "first": integer // Random number is greater than or equal to this value. By default: 0.
  "last": integer  // Random number is less than or equal to this value
}
```

Response

```
integer          // Random number
```

Request Example

```
// ->
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "method": "rand",
  "params": {
    "first": 1,
    "last": 5
  }
}

// <-
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "result": 4
}
```

"method": "sample"

Random selection from the list.

Params

```
{
  "enum": [integer] //
  "limit": integer  // . 1.
}
```

Response

```
[integer]      //
```

Request Example

```
// ->
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "method": "rand",
  "params": {
    "enum": [1, 3, 5, 7, 9],
    "limit": 2
  }
}

// <-
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "result": [3, 7]
}
```

"method": "shuffle"

Random list shuffle.

Params

```
{
  "enum": [integer]    //
}
```

Response

```
[integer]              //
```

Request Example

```
// ->
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "method": "rand",
  "params": {
    "enum": [1, 3, 5, 7, 9]
  }
}

// <-
{
  "jsonrpc": "2.0",
  "id": "d372473a-b0df-47cb-ad64-4b713271cd48",
  "result": [3, 9, 1, 5, 7]
}
```

Additional requirements from Game Provider

- Game provider need to implement Game UI that will be provided by BGaming, it will contain preloader screen, game popups, game UI.
- Game provider need to implement the Replay function on the front end for the support team.

Here are the main points of implementation:

The support employee searches for the necessary round and by clicking on the link in which the round number is transmitted, calls replay method
Runner takes a round from the database and passes it to the game frontend
Frontend starts up and should be able to reproduce the passed result

In this case, the "Back" and "Next" buttons should appear on the frontend, which will allow you to move between the steps within the round

After pressing the "Back" or "Next" button, the step must be played, while the next one should not start until the operator presses the button again

- Game Provider should prepare and share list of game settings and their description.